

SOFTWARE REQUIREMENTS SPECIFICATION

AI Powered CV Ranking & Screening System

TalentMatch Pro

Disclaimer: This repository contains a mock version of one of my freelance project and is shared with the client's permission. To protect privacy and confidentiality, the project's original name and certain sensitive details have been modified, anonymized, or removed before publication.

Document Version	1.0
Status	Initial Draft
Date	June 2026
Client	Entity removed for confidentiality purposes
Prepared By	Arshia Fajar & Client
Project Type	NLP + LLM Automation

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non functional requirements for TalentMatch Pro, an AI powered CV ranking and screening system built for an HR consultancy managing high volume recruitment workflows.

This document serves as the single source of truth for design, development, testing, and delivery of the system.

1.2 Project Background

The client is an xy consultancy serving 20+ corporate clients across industries. Their current process involves manually reading 400 600 CVs per week to produce shortlists, a process that takes 3 full working days per job opening and introduces inconsistency and human fatigue bias.

The client needs an automated system where a recruiter can upload a Job Description (JD) and a batch of CVs and receive a ranked shortlist with AI generated explanations in under 5 minutes.

1.3 Scope

TalentMatch Pro will:

- Parse and extract structured data from PDF and DOCX CVs
- Accept job descriptions as free text or uploaded documents
- Rank candidates using semantic similarity and LLM based evaluation
- Provide explainable, per candidate reasoning for each ranking decision
- Include an optional bias audit flag to surface potential demographic skewing
- Deliver a clean web UI for non technical recruiters
- Export ranked results as downloadable reports

Out of Scope for v1.0:

- ATS (Applicant Tracking System) integration
- Email or calendar automation
- Real time collaboration between multiple recruiters
- Payment or subscription management

1.4 Definitions and Acronyms

Term	Definition
CV / Resume	Curriculum Vitae: a document summarizing a candidate's qualifications
JD	Job Description: defines role requirements, skills, and responsibilities
LLM	Large Language Model: AI model used for language understanding and generation

(TalentMatch is the mock name, actual name has not been used anywhere in this doc)

NLP	Natural Language Processing	field of AI focused on text analysis
Embedding	A numerical vector representation of text used for semantic matching	
RAG	Retrieval Augmented Generation	technique combining search with generation
API	Application Programming Interface	
SRS	Software Requirements Specification	
UI	User Interface	
FastAPI	A Python web framework for building APIs	

2. Overall System Description

2.1 System Context

TalentMatch Pro operates as a standalone web application. Recruiters access it through a browser. The system communicates with an LLM API (Anthropic Claude or open source alternative) and processes documents entirely server side. No candidate data is stored permanently by default.

2.2 Product Perspective

The system is a greenfield build. It will operate independently in v1.0 with API based LLM inference. Future versions may integrate with existing ATS platforms via webhooks.

2.3 User Classes

User Class	Description	Usage Frequency
Recruiter	Primary user. Uploads JDs and CVs, views rankings, downloads reports. Non technical.	High
Admin / Consultant	Manages system configuration, model settings, audit logs. Technical.	Medium
Client (HR Manager)	Receives exported shortlist reports. Does not interact with the system directly.	Low

2.4 Operating Environment

- Backend: Python 3.11+, FastAPI, running on a Linux server or cloud VM
- Frontend: React.js (Vite), served as static files
- LLM: Claude API (primary) or locally hosted Mistral/LLaMA (fallback)
- Embedding Model: sentence transformers (all MiniLM L6 v2 or similar)
- Database: SQLite for v1.0 session storage; PostgreSQL for v2.0
- Deployment: Docker + Docker Compose; deployable on DigitalOcean, AWS EC2, or Railway

2.5 Assumptions and Dependencies

- The client provides CVs in PDF or DOCX format only
- Internet connectivity is required for LLM API calls in the default configuration
- CVs are in English (multi language support deferred to v2.0)
- A valid Anthropic API key will be provided for production use

3. System Architecture

3.1 High Level Architecture

The system follows a 3 layer architecture:

Layer	Components	Responsibility
Layer 1 Ingestion	CV Parser, JD Parser, File Handler	Extracts raw text from uploaded documents
Layer 2 Intelligence	Embedding Engine, Semantic Ranker, LLM Reasoning Module	Scores and explains candidate fit
Layer 3 Presentation	FastAPI REST endpoints, React UI, PDF Report Generator	Serves results to the recruiter

3.2 Data Flow

The end to end data flow for a single screening session:

- Recruiter uploads JD (text or file) + batch of CVs (PDF/DOCX)
- File Handler validates formats and saves to temporary session storage
- CV Parser extracts text from each CV and structures it into JSON
- JD Parser extracts key requirements: skills, experience, role, seniority
- Embedding Engine encodes JD and each CV into vector representations
- Semantic Ranker computes cosine similarity scores for initial ranking
- LLM Reasoning Module takes top N candidates and generates per candidate reasoning
- Results API returns ranked list with scores and reasoning to the frontend
- Recruiter views results, optionally triggers bias audit, downloads report

3.3 Technology Stack

Component	Technology	Purpose	Sprint
CV/PDF Parsing	pdfplumber, python docx	Text extraction from files	Sprint 1
Embedding	sentence transformers	Semantic vector encoding	Sprint 2
Vector Similarity	scikit learn (cosine_similarity)	Candidate JD scoring	Sprint 2
LLM Reasoning	Anthropic Claude API	Explainable shortlisting	Sprint 3
API Backend	FastAPI + Uvicorn	REST endpoints	Sprint 3
Frontend UI	React + Vite + TailwindCSS	Recruiter interface	Sprint 4

File Export	ReportLab or WeasyPrint	PDF report generation	Sprint 5
Containerization	Docker + Docker Compose	Deployment packaging	Sprint 5
Testing	pytest + httpx	Unit and integration tests	Sprint 5

4. Functional Requirements

FR 01: Document Upload

The system shall accept batch upload of CVs in PDF and DOCX format, with a maximum of 100 files per session. The system shall accept a Job Description as free text input or as an uploaded PDF/DOCX file. File size limit per CV: 5 MB.

FR 02: CV Parsing

The system shall extract the following structured fields from each CV where available:

- Full name and contact information (anonymizable)
- Work experience: company names, roles, dates, duration
- Education: degrees, institutions, graduation years
- Skills and technologies (explicit and inferred)
- Certifications and achievements
- Total years of experience (computed)

Unreadable or corrupted files shall return a clear error message without crashing the session.

FR 03: Job Description Analysis

The system shall parse the JD and extract:

- Required skills (must have)
- Preferred skills (nice to have)
- Minimum years of experience
- Education requirements
- Role level (junior, mid, senior, lead)
- Domain/industry keywords

FR 04: Semantic Matching & Ranking

The system shall encode the JD and all CVs using a pre trained embedding model. Cosine similarity scores shall be computed between the JD embedding and each CV embedding. Candidates shall be ranked in descending order of match score. Score shall be displayed as a percentage (0 100%) alongside each candidate.

FR 05: LLM Based Reasoning

For the top N candidates (default: top 20, configurable), the system shall invoke an LLM to generate a structured reasoning block containing:

- Overall fit summary (2 3 sentences)
- Strengths relative to the JD
- Gaps or concerns relative to the JD
- Recommendation tier: Strong Yes / Yes / Maybe / No

Reasoning shall be grounded in the actual CV content hallucinated claims are a critical failure.

FR 06: Bias Audit (Optional Module)

(TalentMatch is the mock name, actual name has not been used anywhere in this doc)

The system shall include an optional bias audit that detects and flags potential demographic signals in CVs including name based gender/ethnicity inference, graduation year (age proxy), and university tier bias. Flagged signals are surfaced as warnings, not filters. The recruiter may dismiss or note each flag.

FR 07: Results Interface

The system shall display ranked candidates in a sortable table with: rank, name, match score, recommendation tier, and a collapsible reasoning panel. The recruiter shall be able to manually override rankings and add notes per candidate.

FR 08: Export

The system shall generate a downloadable PDF shortlist report containing: session metadata (JD title, date, number of CVs screened), ranked candidate table, and per candidate reasoning summaries.

5. Non-Functional Requirements

5.1 Performance

- CV parsing: less than 2 seconds per document
- Embedding + similarity ranking for 100 CVs: under 30 seconds
- LLM reasoning for top 20 candidates: under 90 seconds (sequential API calls)
- Total end to end latency for 100 CVs: under 3 minutes

5.2 Scalability

- v1.0 targets single user or small team usage (1-5 concurrent sessions)
- Architecture shall support horizontal scaling in v2.0 via task queues (Celery + Redis)

5.3 Reliability

- System shall handle malformed PDFs gracefully with per file error reporting
- LLM API failures shall trigger a fallback: display score only without reasoning, with a clear UI indicator
- Uptime target: 99% during business hours (for cloud deployment)

5.4 Usability

- A non-technical recruiter shall complete a full screening session with zero training
- UI shall display progress indicators during long running operations
- All error messages shall be human readable, not stack traces

5.6 Maintainability

- Codebase structured as modular components (parser, embedder, ranker, reasoner, API, UI)
 - Each module independently testable
 - LLM provider shall be swappable via a config flag (Claude / OpenAI / local model)
-

6. Sprint Plan & Delivery Milestones

The project is structured into 6 sprints of 1 2 weeks each. Each sprint produces a working, testable deliverable not just code.

Sprint	Theme	Deliverables	Timeline
Sprint 1	Data Foundation	CV parser (PDF+DOCX), JD text extractor, synthetic dataset of 30 CVs + 3 JDs, data schema defined, all outputs validated as JSON	Week 1 2
Sprint 2	Ranking Engine	Embedding pipeline with sentence transformers, cosine similarity scorer, ranked output for a batch of CVs vs a JD, evaluation on synthetic data	Week 3 4
Sprint 3	LLM Reasoning Layer	Claude API integration, structured reasoning prompt with output schema, per candidate reasoning JSON, hallucination guard (grounding check)	Week 5 6
Sprint 4	API + Basic UI	FastAPI backend with upload/rank/results endpoints, React frontend: upload form, ranked results table, collapsible reasoning panels	Week 7 8
Sprint 5	Export + Bias Audit	PDF report generator, bias signal detection module, UI flags for bias audit results, full session flow working end to end	Week 9 10
Sprint 6	Polish + Deployment	Docker packaging, environment config, README + demo video, case study write up (the deliverable your clients actually read)	Week 11 12

7. Sprint 1 Detailed Task Breakdown

Project folder structure:

```
| talentmatch pro/
|
| ├── data/
| | ├── sample_cvs/
| | └── sample_jds/
|
| ├── src/
| | ├── parser/
| | | ├── __init__.py
| | | ├── cv_parser.py      # Task 1.2
| | | └── jd_parser.py     # Task 1.4
| | └── utils/
| |   └── file_handler.py  # Task 1.3
|
| ├── tests/
| | └── test_parser.py      # Task 1.5
|
| ├── notebooks/
| | └── sprint1_exploration.ipynb
|
| ├── requirements.txt
| └── README.md
```

Task 1.2 CV Parser (3 4 hours)

src/parser/cv_parser.py. This module takes a file path (PDF or DOCX) and returns a structured Python dictionary.

Required output schema:

- name: str candidate's full name (or "Unknown" if not found)
- email: str | None
- phone: str | None
- skills: list[str] extracted skill keywords
- experience: list[dict] each entry: {company, role, duration, description}
- education: list[dict] each entry: {degree, institution, year}
- total_years_experience: float computed from experience dates
- raw_text: str full extracted text (used by the embedding engine in Sprint 2)

Acceptance criteria: run the parser on 5 test CVs. Each returns a valid dict with no crashes. Raw text is non empty for all 5.

Task 1.3 File Handler (1 hour)

src/utils/file_handler.py. A utility that: validates file extension (.pdf, .docx only), checks file size (max 5 MB), routes the file to the correct parser, returns a ParseResult object (success/failure + data or error message).

Task 1.4 JD Parser (2 hours)

src/parser/jd_parser.py. Accepts JD as raw text string. Uses simple NLP (keyword matching + regex) to extract:

- required_skills: list[str]
- preferred_skills: list[str]
- min_experience_years: int | None
- role_level: str junior / mid / senior / lead / unknown
- domain_keywords: list[str]

Note: This does not use an LLM. Pure rule based extraction is fine for Sprint 1. LLM powered JD analysis is Sprint 3.

Task 1.5 Synthetic Data + Tests (2 hours)

Create 10 synthetic CVs (you can write these as simple text files, then convert to PDF using any tool). Make them realistic mix of good fits, partial fits, and weak candidates for at least 1 test JD (e.g. a Data Analyst role).

Write tests/test_parser.py with pytest:

- test_pdf_parsing_returns_dict: asserts output is a dict with all required keys
- test_raw_text_not_empty: asserts raw_text field is non empty
- test_invalid_file_returns_error: asserts FileHandler returns error for .jpg file
- test_jd_parser_extracts_skills: asserts at least 3 skills extracted from a sample JD

Sprint 1 is complete when: all 4 tests pass and you can run the parser on your 10 synthetic CVs and print their structured output.

8. Evaluation & Success Criteria

8.1 Per Sprint Definition of Done

A sprint is complete only when:

- All tasks have working, committed code in the GitHub repo
- Tests pass (where applicable)
- A short progress note is written in the repo's README or a PROGRESS.md file
- The deliverable has been demoed (even to yourself - run it, see output)

8.2 Project Success Metrics

Metric	Target
Ranking accuracy	Top 3 ranked candidates are considered strong fits by a human recruiter in 80%+ of test sessions
Reasoning quality	LLM reasoning cites specific CV content in 90%+ of cases (no hallucination)
Processing speed	100 CVs ranked and reasoned in under 3 minutes
Bias detection	System flags at least 2 types of demographic signals when present in test data
Usability	A non technical person can complete a session without instructions
Portfolio quality	Project includes README, case study write up, and a working demo (video or live URL)

9. Risk and Mitigation:

Risk	Level	Mitigation	Sprint
LLM hallucination	High	Grounding check: verify all LLM claims reference actual CV text	Sprint 3
PDF parsing failures	Medium	Fallback to raw pdfminer extraction; per file error handling	Sprint 1
API cost overrun	Medium	Cache LLM results per session; limit reasoning to top 20 candidates	Sprint 3
Scope creep	High	Strictly follow sprint plan; all new features go into a backlog for v2.0	All
Dropping the project	High	Weekly check in with technical lead (this document is your commitment)	All

10. Document Revision History

Version	Date	Changes	Author
1.0	June 2026	Initial SRS all sections drafted	Consultant